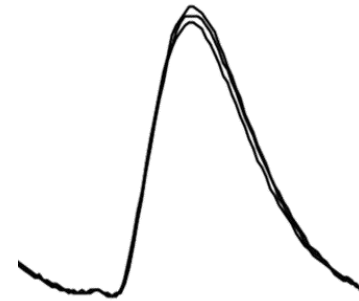
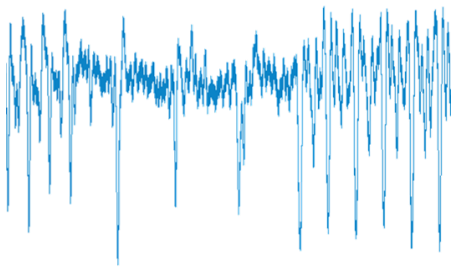


Code:

Software Side Channels

David Oswald



Goals of this lecture

After this lecture, you will ...

- Understand sources of side-channel leakage in software, especially caches
- See again how small leakages can lead to very efficient attacks
- Have an idea about the need for and cost of countermeasures

A short history of side-channel attacks

1996: Timing attacks (Kocher)

1998: Differential Power Analysis and Simple Power Analysis
by Kocher et al.

... various improvements and countermeasures ...

2005: “Cache-timing attacks on AES” (Bernstein)

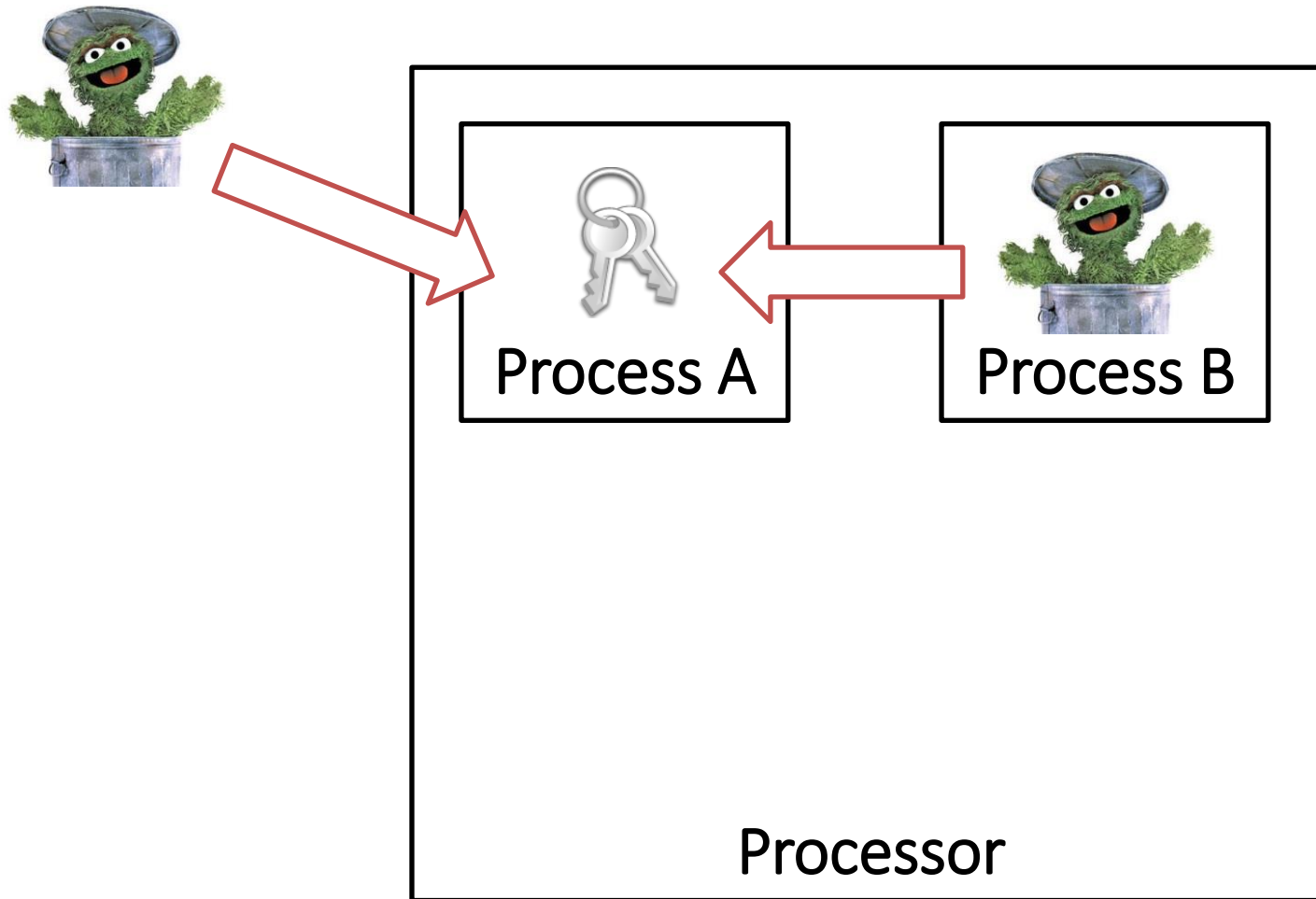
... some more software side-channel attacks ...

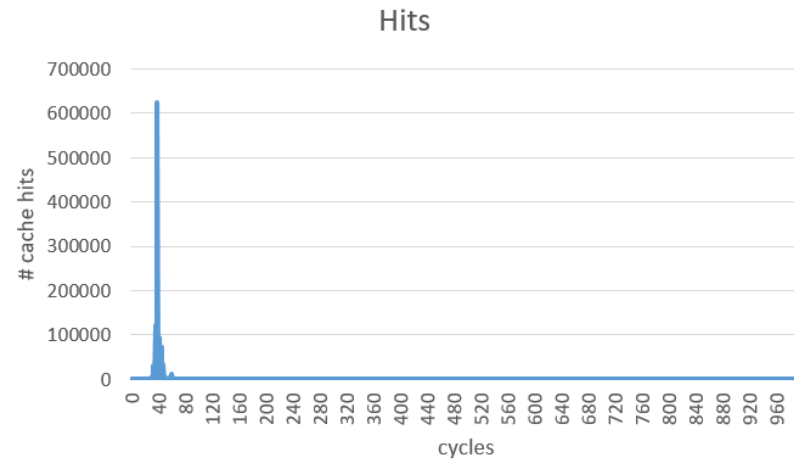
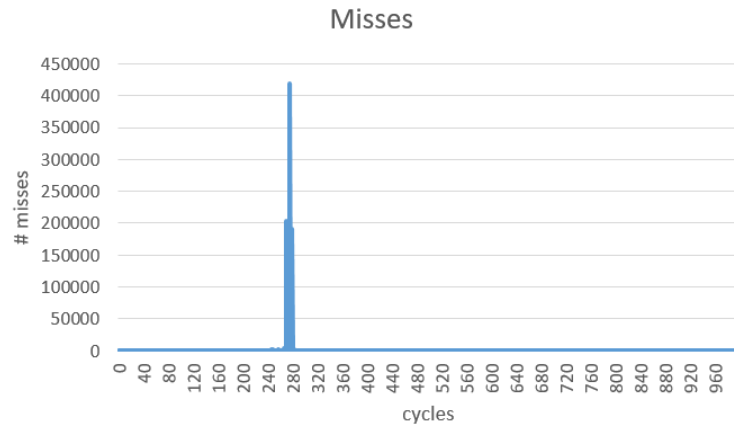
2014: “FLUSH+RELOAD: A High Resolution, Low Noise, L3 Cache
Side-Channel Attack” (Yarom et al.)

2018: Spectre / Meltdown

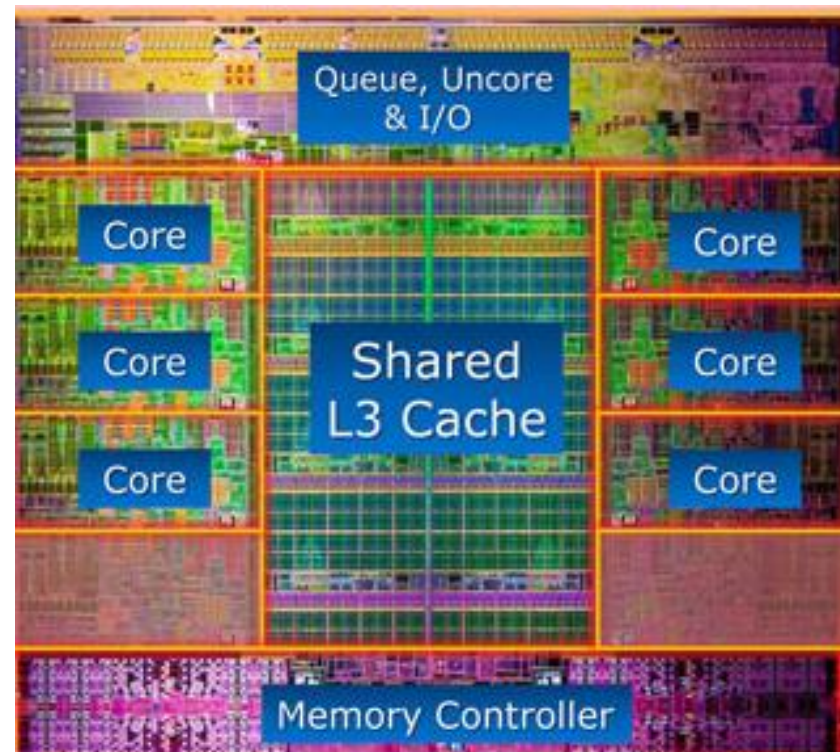
... many more software side-channel attacks ...

Attack setup





Cache attacks



An initial question...

Is this code constant time?

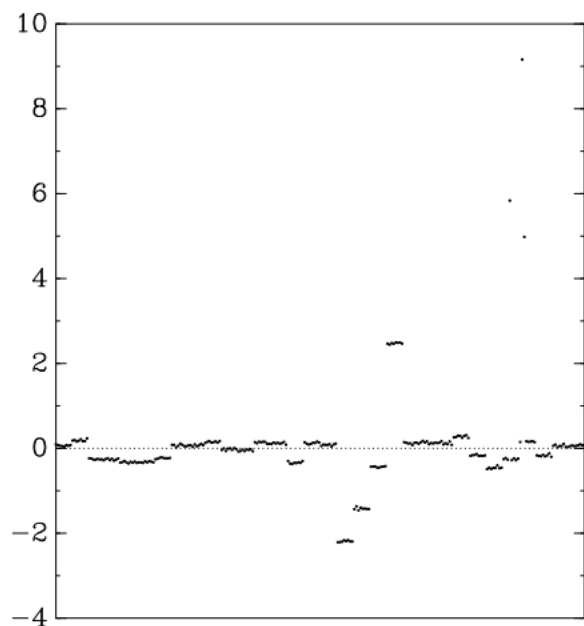
$$B_0 = \text{sbox}[x[0] \wedge k[0]]$$

No (if there is cache)....

Bernstein's attack (2005)

<https://cr.yp.to/antiforgery/cachetiming-20050414.pdf>

- Execution time of OpenSSL AES depends on plaintext (and key) due to cache
- This could be exploited even over the network to extract the key in 160 min

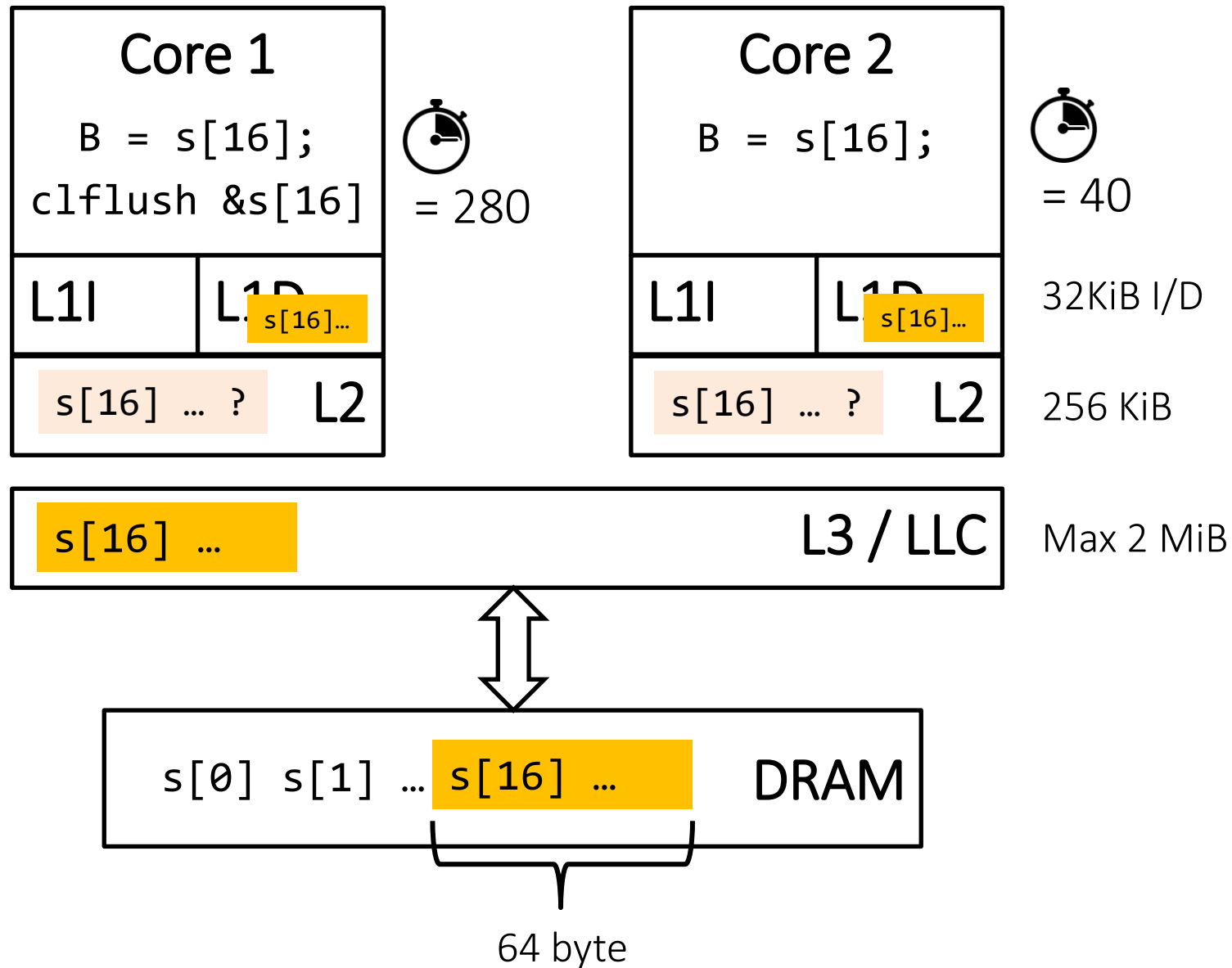


“RAM cache hits can produce timing characteristics,” Kocher wrote in 1996 in [16, Section 11], after explaining in detail how to extract secret keys from timing leaks in another operation. A decade later, we still have cryptographic software—in particular, AES libraries—whose timing obviously varies with, among other things, input-dependent RAM cache hits. How did this happen?

Fig. 5.3. How $n[15]$ affects OpenSSL AES timings for $k = 0$ on a Pentium III inside the targeted server. The horizontal axis is the choice of $n[15]$, from 0 through 255. The vertical axis is the average number of cycles for about 2^{22} 800-byte packets with that choice of $n[15]$, minus the average number of cycles for all choices of $n[15]$.

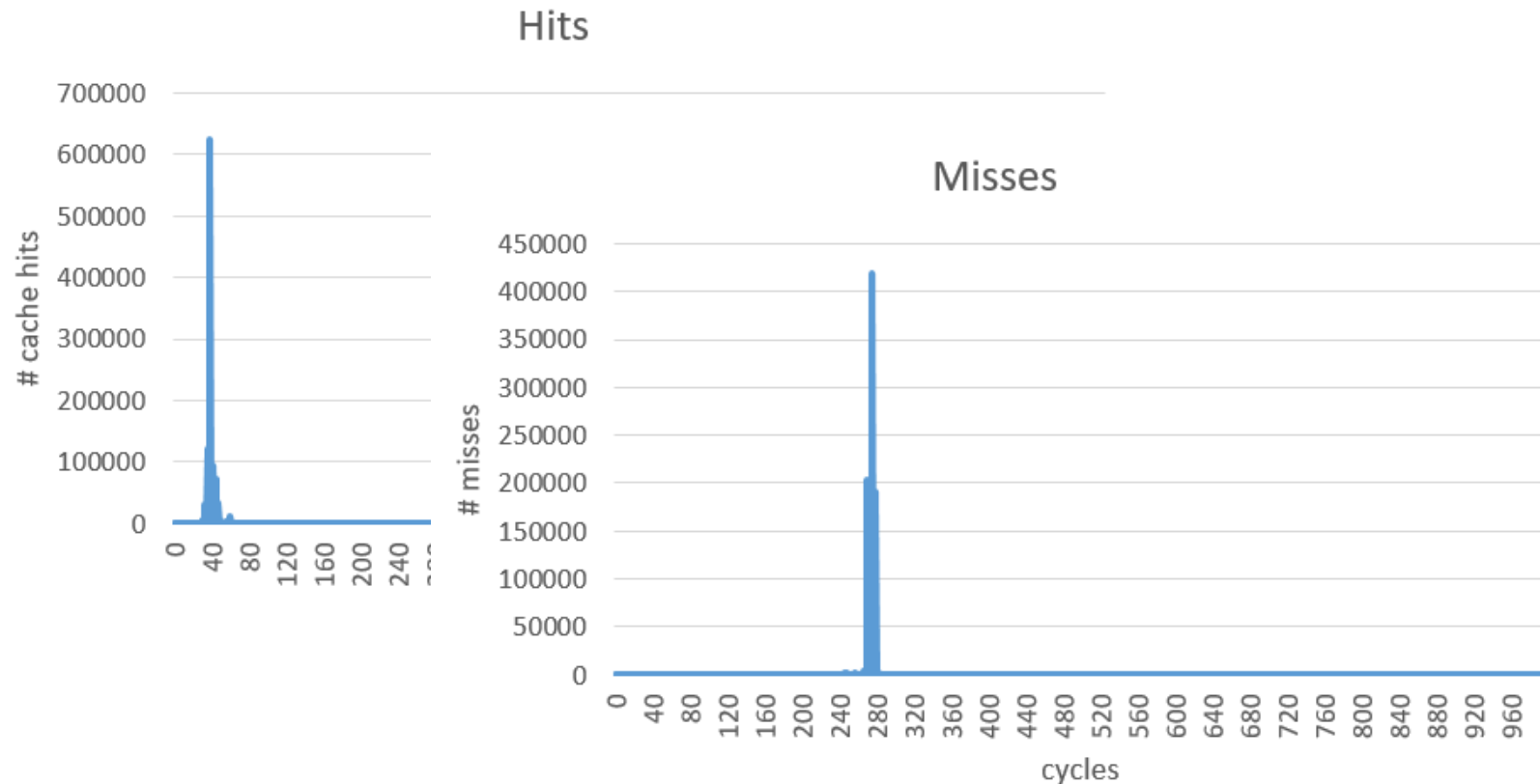
A typical cache hierarchy (Intel Kaby Lake)

https://en.wikichip.org/wiki/intel/microarchitectures/kaby_lake



Cache vs. memory

Fetching from cache (“hit”) is faster than from DRAM (“miss”)



Live-Demo

“Anything that can go wrong,
will go wrong”

Cache timing measurement

FLUSH+RELOAD (2014)

<https://eprint.iacr.org/2013/448.pdf>

- Assumes that the adversary has local, **non-privileged** code execution
- Targets LLC (L3) cache, i.e. it works across cores (previous attacks often same-core)
- Basic steps of attacker “probe”:
 - Flush the cache
 - Run the victim
 - Measure access times to victim memory

The basic FLUSH+RELOAD primitive

<https://eprint.iacr.org/2013/448.pdf>

```
1  int probe(char *adrs) {
2      volatile unsigned long time;
3
4      asm __volatile__ (
5          " mfence                \n"
6          " lfence                \n"
7          " rdtsc                 \n"
8          " lfence                \n"
9          " movl %%eax, %%esi      \n"
10         " movl (%1), %%eax       \n"
11         " lfence                \n"
12         " rdtsc                 \n"
13         " subl %%esi, %%eax       \n"
14         " clflush 0(%1)          \n"
15         : "=a" (time)
16         : "c" (adrs)
17         : "%esi", "%edx");
18     return time < threshold;
19 }
```

Run in a loop on
target address(es)
parallel to victim



T0



T1

*(addr)

clflush addr

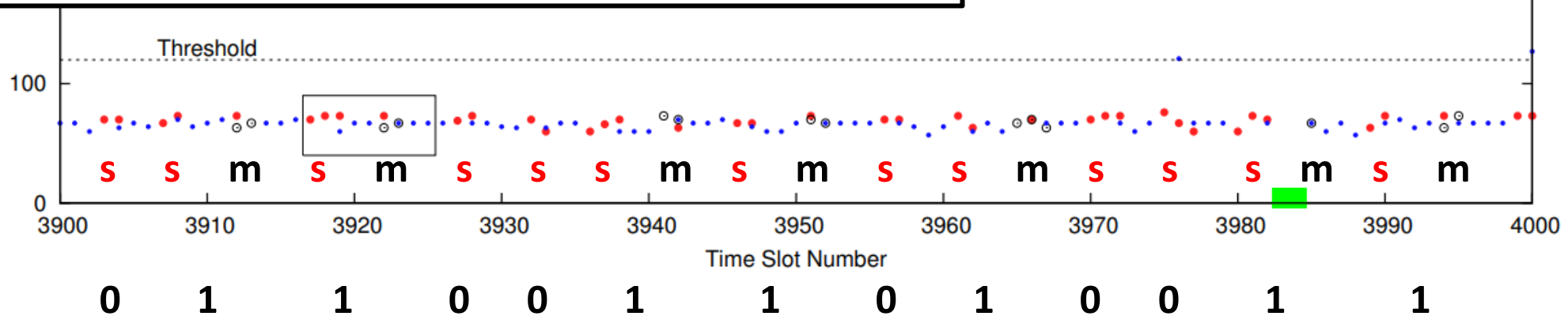
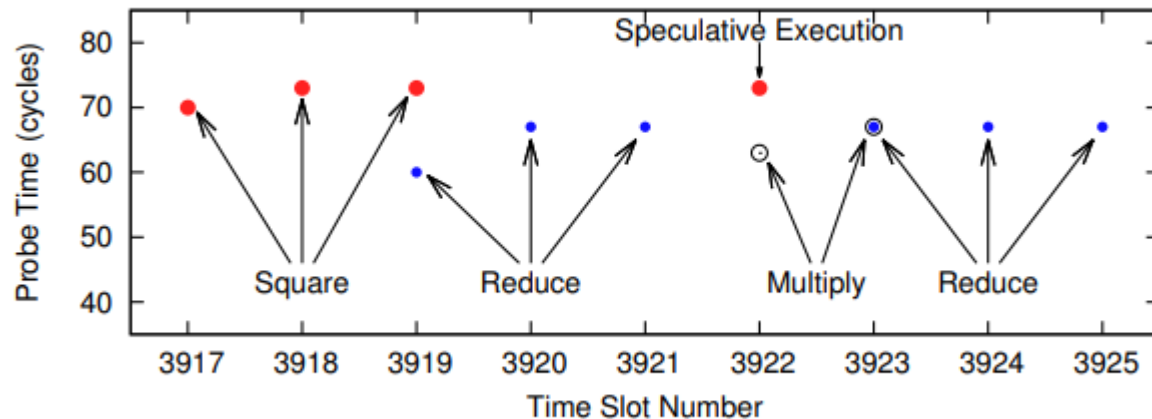
T1 - T0 < tresh

Figure 4: Code for the FLUSH+RELOAD Technique

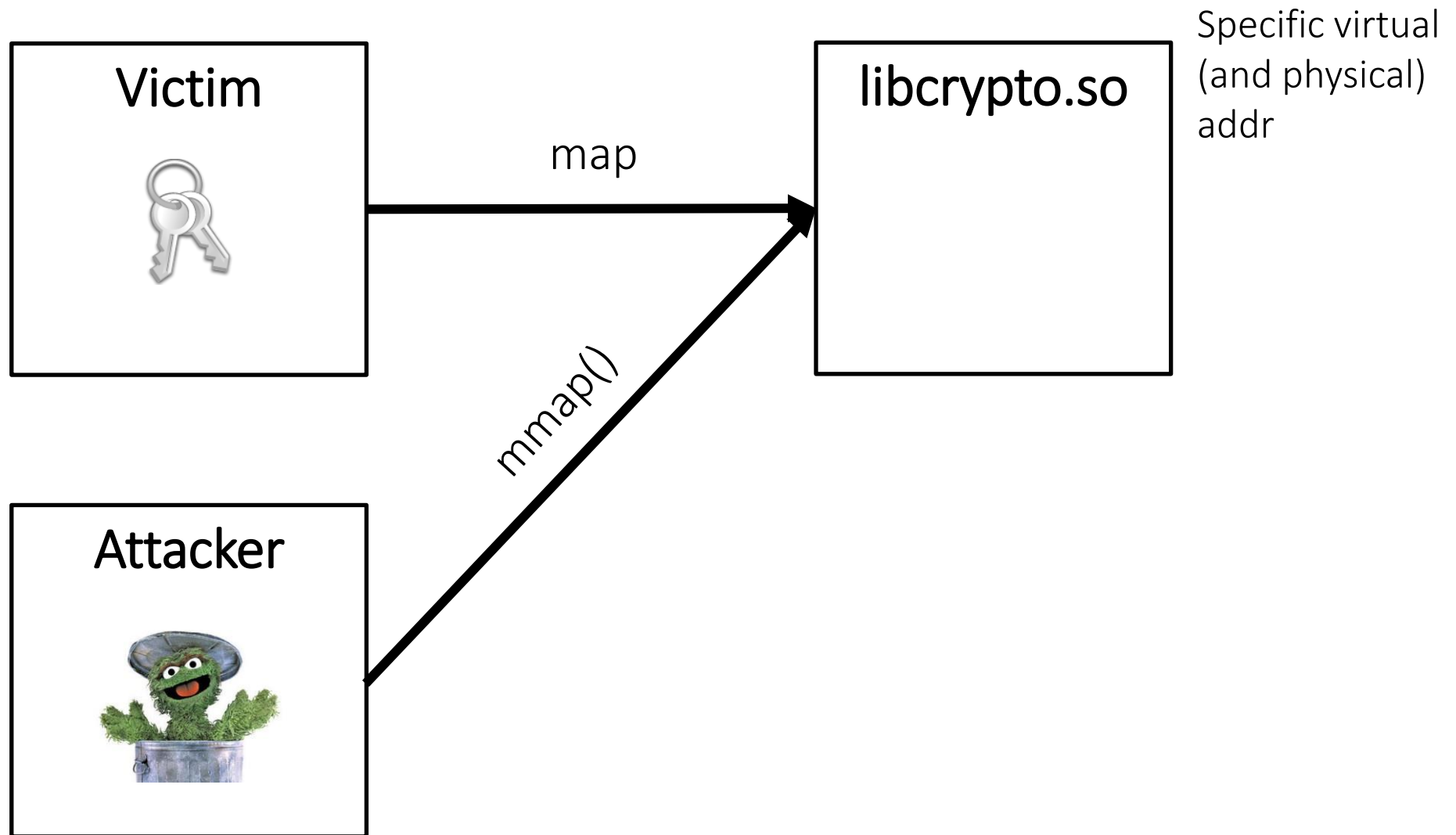
Example: RSA in GPG

<https://eprint.iacr.org/2013/448.pdf>

probe(**s**quare)
probe(multiply)
probe(**m**odulo)



How to find the probe addresses?



FLUSH+RELOAD

- Has been used in various ways after its discovery
- A building block of Spectre/Meltdown
- There are other variants (e.g. Flush+Flush)

- Further reading:

<https://eprint.iacr.org/2013/448.pdf>

https://github.com/IAIK/cache_template_attacks

https://seedsecuritylabs.org/Labs_16.04/System/Meltdown_Attack/

Countermeasures?



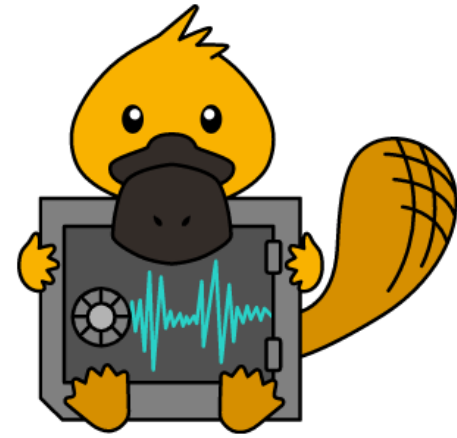
How to prevent this?

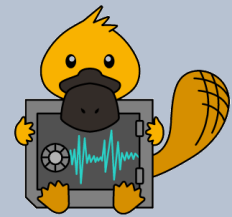
- Do **not** use secret-dependent memory lookups (avoid `a = b[f(k)]`)
- This is really **hard** in general
- Most solutions specific to crypto code, e.g. bitslicing and/or specific coding style
- One reason why x86 has dedicated AES hardware support (`aesenc` etc)

PLATYPUS

**WITH GREAT POWER COMES GREAT
LEAKAGE**

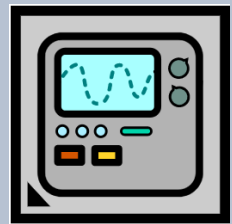
Beyond the
cache ...





Other side channels

- Apart from the cache, there are many more possible software side channels
- One example is the Platypus attack:
<https://platypusattack.com/>
- Uses Intel's Running Average Power Limit (RAPL) functions to measure **power consumption** of target operations
- From there, we're back to classical side-channel attacks



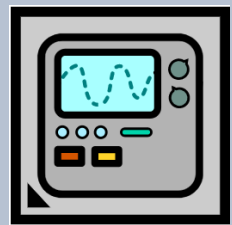
RAPL: An oscilloscope in your CPU

RAPL on Linux

There are currently **three** ways to read RAPL results using the Linux kernel:

- Reading the files under `/sys/class/powercap/intel-rapl/intel-rapl:0` using the powercap interface. This requires no special permissions, and was introduced in Linux 3.13
- Using the `perf_event` interface with Linux 3.14 or newer. This requires root or a `paranoid` less than 1 (as do all system wide measurements with -a) `sudo perf stat -a -e "power/energy-cores/" /bin/ls`
Available events can be found via `perf list` or under `/sys/bus/event_source/devices/power/events/`
- Using raw-access to the underlying MSR's under `/dev/msr`. This requires root.

- Measure acc. energy consumption E_0
- Run target
- Measure acc. energy consumption E_1
- Use difference for $E_1 - E_0$

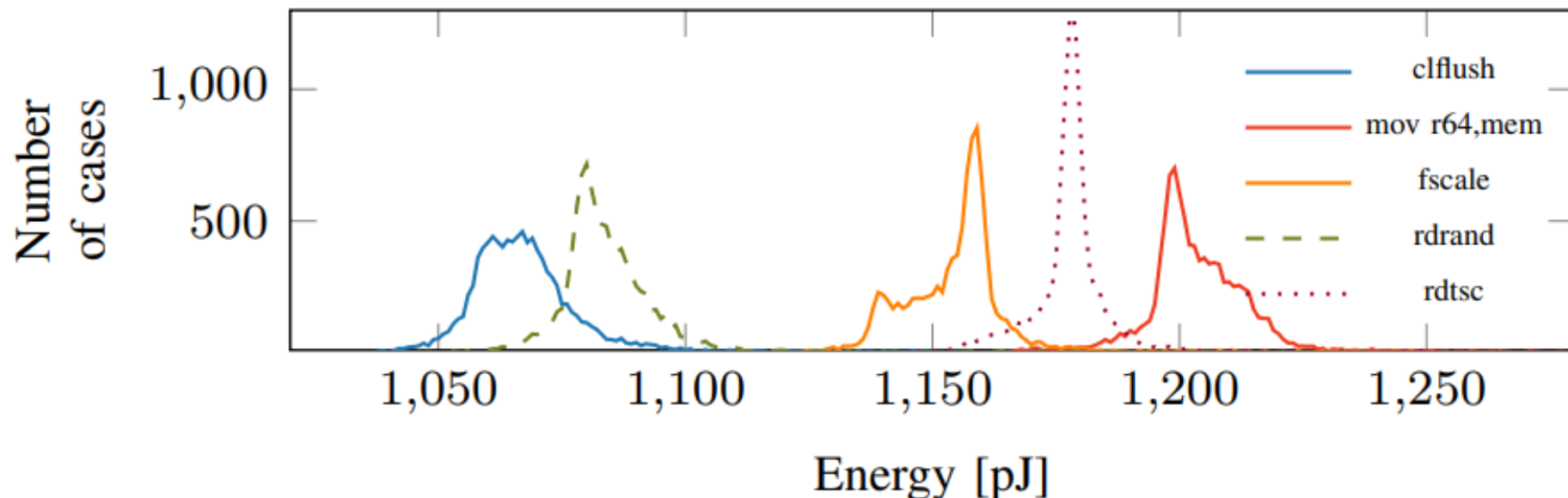


RAPL: An oscilloscope in your CPU

RAPL on Linux

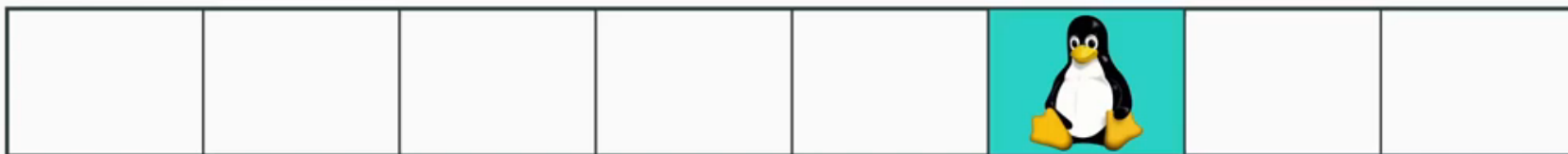
There are currently **three** ways to read RAPL results using the Linux kernel:

- Reading the files under `/sys/class/powercap/intel-rapl/intel-rapl:0` using the powercap interface. This requires no special permissions, and was introduced in Linux 3.13
- Using the `perf_event` interface with Linux 3.14 or newer. This requires root or a `paranoid` less than 1 (as do all system wide measurements with -a) `sudo perf stat -a -e "power/energy-cores/" /bin/ls`
Available events can be found via `perf list` or under `/sys/bus/event_source/devices/power/events/`
- Using raw-access to the underlying MSRs under `/dev/msr`. This requires root.



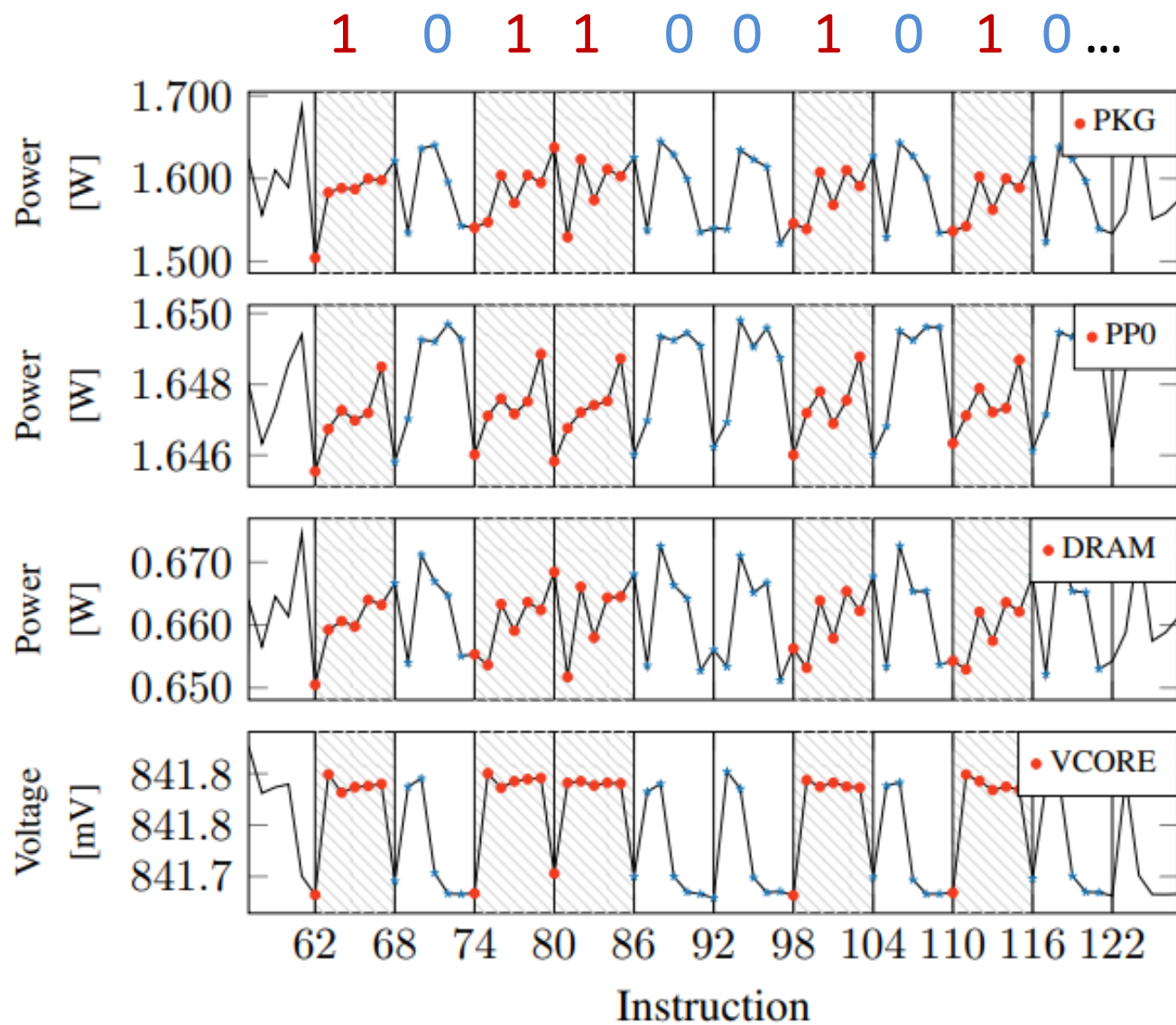
Example attacks

- RAPL is exposed by the kernel to unprivileged users – so we can attack kernel level code (e.g. KASLR)



Example attacks

- RAPL is exposed by the kernel to unprivileged users – so we can attack kernel level code (e.g. KASLR)
- For Intel SGX (remember Plundervolt), we can single-step the enclave and get precise results





Platypus: Summary

- Platypus can break things that cache timing can't
- Power consumption depends on processed data, so this applies to cache-timing secure code (including AES-NI)
- Was fixed by
 - Removing unprivileged access (protects against user-to-kernel attacks)
 - Microcode update to protect SGX

Take-home points

- Cache attacks break any **secret-dependent memory access** and will continue to work
- Defenses are **non-trivial** and are therefore mainly applied to crypto code
- Flush+Reload is a very generic attack technique
- Beyond that, there are other side channels
- We'll see Flush+Reload again in the lecture on transient execution attacks like Meltdown

Thanks!

Questions / discussion?

d.f.oswald@bham.ac.uk